HPC-2026

MPI Programming Tutorial

Message Passing • Blocking & Non-Blocking Communication • Process Topologies

- Explain MPI processes, ranks, communicators, and message passing.
- Write and execute a basic MPI program.
- Exchange data between processes using MPI_Send and MPI_Recv.
- Measure elapsed time using MPI_Wtime and compare message sizes.
- Use MPI_Isend, MPI_Irecv, and MPI_Wait to explore possible computation–communication overlap.
- Implement ring and two-dimensional grid communication patterns.
- Interpret experimental results and explain limitations.

2. Reference Reading

Read Chapter 6, "Programming Using the Message-Passing Paradigm," from Introduction to Parallel Computing by Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar.

Official reference: MPICH user guides and tutorials are available from the MPICH documentation website.

3. MPI Concepts

MPI (Message Passing Interface) is a standard for writing parallel programs whose processes communicate by exchanging messages. Each process has its own rank within a communicator.

Concept	Meaning
Process	An independent execution unit running part of the MPI program.
Rank	A unique integer identifier for a process within a communicator.
MPI_COMM_WORLD	The default communicator containing the processes launched for the MPI job.
Message passing	Transferring data from one process to another.
Blocking operation	A call returns only when its operation-specific completion conditions are met.
Non-blocking operation	A call initiates an operation and returns a request that can be waited on or tested.

4. Installation, Compilation, and Execution

4.1 Install MPICH on Ubuntu/Debian

```
sudo apt update
sudo apt install mpich
```

Check that the compiler wrapper is available:

```
mpicc --version
```

4.2 Compile and run

```
mpicc hello_mpi.c -o hello_mpi
mpirun -np 4 ./hello_mpi
```

The command -np 4 launches four MPI processes. Output order may vary because processes can run concurrently.

5. Problem 1 — Hello World with MPI

Objective: Learn the basic structure of an MPI program.

Write a program that prints each process rank and the total number of processes. Use the following functions:

- MPI_Init(&argc, &argv) — initialize MPI.
- MPI_Comm_rank(MPI_COMM_WORLD, &rank) — obtain the current process rank.
- MPI_Comm_size(MPI_COMM_WORLD, &size) — obtain the total number of processes.
- printf(...) — print the rank and size.
- MPI_Finalize() — finish MPI execution.

Program structure (not a complete solution):

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int rank, size;

    /* Initialize MPI */
    /* Find rank and communicator size */
    /* Print rank and size */
    /* Finalize MPI */

    return 0;
}
```

Write the complete program yourself, compile it, and run it with two and four processes. Explain why the same program prints different ranks.

6. Problem 2 — Blocking Point-to-Point Communication

Objective: Send an integer array from Process 0 to Process 1 using MPI_Send and MPI_Recv, then measure elapsed time.

6.1 Communication flow

Process 0 (sender) ——— array message ———> Process 1 (receiver)

6.2 Function signatures

```
MPI_Send(buffer, count, datatype, destination, tag, communicator);
MPI_Recv(buffer, count, datatype, source, tag, communicator, &status);
```

Key parameters: buffer is the data address; count is the number of elements; datatype may be MPI_INT; destination/source are process ranks; tag identifies the message; communicator identifies the process group.

6.3 Tasks

1. Process 0 initializes an integer array and sends it to Process 1.
2. Process 1 receives the array and prints its first and last elements.
3. Use MPI_Wtime() to measure elapsed time for the transfer.
4. Repeat the experiment for 1 MB, 10 MB, and 50 MB messages.
5. Run multiple trials and calculate the average time.
6. Plot message size on the X-axis and average communication time on the Y-axis.

6.4 Message-size calculation

Assuming sizeof(int) = 4 bytes and using decimal MB (1 MB = 1,000,000 bytes):

Target message size	Number of 4-byte integers
1 MB	250,000
10 MB	2,500,000
50 MB	12,500,000

One million 4-byte integers occupy about 4 MB (decimal), not 1 MB. Check sizeof(int) on the system if exact sizes matter.

6.5 Timing guidance

```
MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();
/* perform the communication and ensure receiver completion */
double elapsed = MPI_Wtime() - start;
```

MPI_Wtime returns elapsed wall-clock time in seconds on the calling process. Its clock is local to each process, so do not subtract timestamps taken on different processes. For a complete-transfer measurement, coordinate the receiver and sender so the timing includes the intended transfer and receipt.

Message Size	Trial 1 (s)	Trial 2 (s)	Trial 3 (s)	Average (s)
1 MB				
10 MB				
50 MB				

Analysis questions:

- Why does transfer time generally increase as the message grows?
- Does doubling the message size always double the time? Why or why not?
- What influence do latency, bandwidth, memory copying, and system load have?
- What does blocking mean for the calling process? Does MPI_Send returning always mean the receiver has processed the message?

7. Problem 3 — Non-Blocking Communication and Overlap

Objective: Replace MPI_Send/MPI_Recv with MPI_Isend/MPI_Irecv and investigate whether independent computation can overlap with communication.

7.1 Function signatures

```
MPI_Isend(buffer, count, datatype, destination, tag, communicator, &request);
MPI_Irecv(buffer, count, datatype, source, tag, communicator, &request);
MPI_Wait(&request, MPI_STATUS_IGNORE);
```

7.2 Tasks

7. Initiate the send on Process 0 with MPI_Isend.
8. Initiate the receive on Process 1 with MPI_Irecv.
9. Perform a dummy computation loop while communication is outstanding.
10. Call MPI_Wait before reusing the send buffer or accessing the received data.
11. Measure elapsed time for 1 MB, 10 MB, and 50 MB messages.
12. Compare results with Problem 2 using the same message sizes and a comparable workload.

Important correctness rules:

- Do not modify or reuse a send buffer before the associated send request completes.
- Do not read the received data as complete before the receive request completes.
- Non-blocking calls permit overlap but do not guarantee it or guarantee faster execution.
- Use independent computation; work that depends on the incoming message cannot be performed before the receive completes.

Message Size	Blocking Time (s)	Non-Blocking Time (s)	Observation
1 MB			
10 MB			
50 MB			

Discussion: Did the dummy computation overlap with communication on your system? What factors could explain the results?

8. Problem 4 — Ring Communication Topology

Objective: Connect processes in a logical ring. Each process sends a message to its next neighbor and receives a message from its previous neighbor.

```
next = (rank + 1) % size;
previous = (rank - 1 + size) % size;
```

Rank (4 processes)	Previous	Next
0	3	1
1	0	2
2	1	3
3	2	0

Tasks:

- Write a program that sends a message to the next neighbor and receives from the previous neighbor.
- Print the sender rank and received value at each process.
- Test with at least four processes.
- Explain how the send/receive order can cause deadlock. Consider MPI_Sendrecv or a carefully designed ordering.

9. Problem 5 — Two-Dimensional Grid Topology

Objective: Arrange processes in a 2D grid and exchange messages with valid neighbors in the up, down, left, and right directions.

Example: a 2 × 2 grid.

Process 0	Process 1
Process 2	Process 3

For a row-major layout with C columns:

```
row = rank / C;
column = rank % C;
```

For a 2 × 2 grid:

- Process 0: right = 1, down = 2.
- Process 1: left = 0, down = 3.
- Process 2: up = 0, right = 3.
- Process 3: up = 1, left = 2.

Tasks:

13. Determine the row and column of each process.
14. Identify valid neighbors and handle boundaries correctly.
15. Exchange a message with each valid neighbor.
16. Print each process rank and the ranks of its neighbors.
17. For an extension, investigate MPI_Cart_create and MPI_Cart_shift.

10. Submission Requirements

- Source code for each completed problem, with comments.
- Compilation and execution commands.
- Sample output showing successful execution.
- Timing results and graph for blocking communication.
- Timing comparison and analysis for non-blocking communication.
- A brief explanation of ring and 2D grid topologies.
- Answers to the analysis questions, based on your own observations.

11. Troubleshooting Checklist

- Compile with mpicc, not plain gcc.
- Check that the number of processes matches the ranks used in the program.
- Ensure the sender and receiver use compatible counts, datatypes, tags, and communicators.
- Check array allocation and avoid exceeding available memory for 50 MB messages.
- If the program hangs, inspect matching sends/receives and possible deadlock.
- Do not access non-blocking receive data before MPI_Wait or another valid completion check.
- Repeat timing runs and report measured results; never invent or estimate experimental values.